

PROJECT MANUAL — AlgoChowk NIFTY Recovery Assignment

Everything, explained, so you can talk about this cold.

This is not a submission file. It's for you — read it until you could explain any line of it to someone grilling you in an interview.

PART 1 — THE ONE-SENTENCE VERSION

Question asked: Does NIFTY bounce back after a big one-day drop? **Answer found:** No — not in any way that survives scrutiny. Whatever weak pattern shows up is indistinguishable from noise, and it doesn't even point the same direction in different time periods.

If you remember nothing else, remember that sentence and why we're confident in it (multiple testing check + out-of-sample check, both explained below).

PART 2 — THE FULL STORY, IN ORDER

Step 1: Get the data

File: `src/get_data.py`

We needed NIFTY's daily price history: Open, High, Low, Close for every trading day. We used `yfinance`, a free Python library that pulls data from Yahoo Finance, ticker `^NSEI` (the `^` prefix marks it as an index, not a stock).

Why Yahoo Finance and not NSE's own site? NSE's official archives require session cookies/headers that are annoying to script reliably. Yahoo is free, needs no API key, and is a source anyone reviewing your work can independently re-check. That's what "credible and documented" means in the brief — it doesn't have to be the fanciest source, just one you can name and defend.

What the script actually does: calls `yf.download("^NSEI", period="max", interval="1d")`, which returns the entire available history as a pandas DataFrame, then saves it to `data/nifty_raw.csv`.

Result: **4,663 rows, 2007-09-17 to 2026-09-18.**

Step 2: Validate the data

File: `src/inspect_data.py`

Before trusting any dataset, you check it's not garbage. We checked:

- **Duplicate dates** — same day appearing twice (would double-count an event). Found: 0.
- **Missing values** — a blank Open/High/Low/Close (breaks every calculation downstream). Found: 0.
- **Date ordering** — file should run oldest to newest with no gaps going backward. Confirmed monotonic.
- **OHLC internal consistency** — for any single day, `High` must be the largest of the four numbers and `Low` the smallest (a day's high can't be lower than its own close). If this fails, the data source has a scraping bug. Found: 0 violations.
- **Rows per year** — sanity-check against the roughly 245-250 trading days a normal year has. Confirms no year has a chunk of data silently missing.
- **Daily returns** — `(Close_today / Close_yesterday) - 1`. This is the single most important derived number in the whole project — everything else is built on it.
- **Suspicious single-day moves** — anything with `|return| > 10%` flagged for a human to look at, not

auto-deleted. We found 3: the 2008 financial crisis crash, a 2009 post-election rally (India's market actually hit a circuit breaker and paused trading that day because the move was so large), and the March 2020 COVID crash. All three are real, well-documented events — so we kept them. **You never delete an extreme value just because it's extreme; you only delete it if you can show it's a data error.**

Step 3: Define “significant fall” and “recovery” — mathematically

File: `src/config.py` (the numbers) + `src/event_engine.py` (the logic)

The assignment doesn't hand you a definition — you have to invent one and defend it. Here's ours, and the reasoning behind every piece:

“Significant fall” → an EVENT

```
Event on day t if (Close_t / Close_{t-1}) - 1 ≤ -3%
```

In plain English: if today's closing price is at least 3% lower than yesterday's closing price, today counts as an event day.

Why -3% and not some other number? There's no universally correct answer — that's explicitly stated in the brief. -3% is large enough to be rare (roughly 1-2% of all trading days) but common enough to get a usable sample size. We didn't just pick one number and hope — we **swept** the threshold from -1.5% to -5% (Step 5 below) specifically to show the conclusion doesn't hinge on this one choice.

Why measure the fall using CLOSE-to-CLOSE, not open, or intraday low?

Close-to-close is the standard, unambiguous way to measure “how much did the index move today” — everyone agrees what a closing price is, whereas intraday low is noisier and harder to define precisely across data sources.

“Recovery” — TWO separate definitions (this is a key design choice)

We deliberately used two different definitions rather than one, because they answer different questions:

1. **Statistical recovery:** the *average* forward return after events is positive, and more than half of events show a positive forward return (the “win rate”). This is the standard finance-research way to ask “is there a pattern here.”
2. **Practical recovery:** the price actually climbs back up to at least its pre-fall level within the holding period. This is what a normal person means when they say “it recovered” — the loss was erased.

Why both matter: it's mathematically possible for the *average* forward return to be positive while almost none of the individual events actually recovered the loss — a few big wins can drag the average up while most events stayed in a hole. This exact thing happened in our data (see Part 3) and it's one of the more sophisticated points in the whole project. Anyone asking “did you just report the number that looked best” — this is your answer: no, we checked it two ways specifically because one can mislead without the other.

Independence / event clustering

A market crash doesn't produce one bad day — it produces several bad days in a row (Jan 2008 has three -3%+ days within a week). If you treat each of those as a separate, independent “recovery test,” you're secretly testing the same crash three times and pretending you have three times the evidence you actually do.

Our rule: an event within 3 trading days of the previous event is marked `is_independent = False`. We report results BOTH ways (all events, and independent-only) so nothing is hidden either direction.

Entry price — WHY `Open` of the NEXT day, not `Close` of the event day

This is a subtle but important point, and a good one to be able to explain: you only *know* today was a -3% day once the market closes. You cannot place a trade at today's closing price using information that only exists at that exact closing moment — in practice you'd need to act a fraction of a second before market close, which isn't realistic. Using the close of the event day itself is called **look-ahead bias** — using information you couldn't actually have had at the time of the decision. The realistic move is: signal confirmed at close of day t , you act at the very next opportunity, which is `Open` on day $t+1$.

Holding period

We tested 1, 3, 5, and 10 trading days — not just one — again so the conclusion doesn't depend on picking a lucky number.

Step 4: Build the event detection + forward-return engine

File: `src/event_engine.py`

Three main functions, each doing one clear job:

- `detect_events()` — scans every day's return, flags the ones $\leq$ threshold, tags which ones are independent vs. clustered.
- `compute_forward_returns()` — for every event, works out the entry price (next day's open), then for each holding period, the exit price (close N days later) and the resulting return. Also checks the "practical recovery" condition (did price reclaim the pre-fall close).
- `compute_baseline_forward_returns()` — does the exact same forward-return calculation, but for *every single day in the dataset*, not just event days. This baseline is essential: NIFTY drifts upward over multi-year periods just from general economic growth, so "positive return after a fall" means nothing unless it beats what a random, non-event day would have returned anyway.

Config-driven, not hardcoded — every threshold/holding period/gap lives in `config.py`, so changing an assumption never means editing the detection logic itself. This was an explicit requirement in the brief ("changing the event threshold or holding period should not require rewriting the research engine") and it's also just good practice.

Step 5: Statistical evidence — is the pattern real or noise?

File: `src/stats_engine.py`

This is the part worth understanding deepest, because it's the intellectual core of the project and the part most likely to get interview questions.

The problem with a plain average

If event-day forward returns average +0.4% and baseline days average +0.1%, is that a real difference or could it easily happen by chance with only ~60-80 events? You need a formal way to answer "how likely is a gap this big, if there's actually no real effect at all?"

Why we used bootstrap + permutation tests instead of just a classic t-test

A classic t-test assumes your data is roughly bell-curve shaped (normal) and each data point is independent of the others. Daily stock returns violate both assumptions: they have "fat tails" (extreme days are more common than a bell curve predicts) and forward returns computed over overlapping windows share data with each other. Using a method that requires these assumptions when they're false can give you a false sense of confidence.

Bootstrap (used for confidence intervals): take your actual sample of returns, randomly draw from it *with replacement* thousands of times (each draw is the same size as your original sample), compute the mean of each fake resample, and look at the spread of those means. That spread IS your confidence interval — built entirely from your actual data's own shape, no bell-curve assumption needed. **In plain terms: "if I had**

gotten a slightly different lucky/unlucky draw of events, how much would my average have wobbled?"

Permutation test (used for the p-value comparing event days vs. baseline days): pool ALL the returns (event days + baseline days) together into one big bucket, and thousands of times, randomly deal that pool back into two fake groups of the same sizes as your real groups. Each time, compute the difference in means between the two fake groups. If your REAL observed difference is bigger than almost all of the FAKE random differences, that's evidence the real labeling (event vs. non-event) actually matters. **In plain terms: "if I randomly relabeled which days were 'events,' how often would I see a gap this big purely by luck?"** The p-value is literally that fraction.

We still reported the classic Welch's t-test too — not because it's better, but because reviewers expect to see it, and stating explicitly why we don't rely on it (the assumptions above) is a stronger answer than pretending it doesn't exist.

What a p-value actually means (get this exactly right if asked)

A p-value of 0.03 means: *"if there truly were no real difference between event days and baseline days, you'd see a gap this large or larger only 3% of the time by random chance."* It does **not** mean "there's a 97% chance the effect is real" — that's a very common misinterpretation. Conventionally, $p < 0.05$ is called "statistically significant," but that's just a convention, not a law of nature.

What we actually found

At the primary -3% threshold, **every p-value was between 0.12 and 0.94** — nowhere close to significant at any holding period. No evidence of a real recovery pattern at this threshold.

Step 6: Robustness sweep — did we just get unlucky with -3%?

File: `src/robustness_sweep.py`

We reran the entire detect → forward-return → stats pipeline across 7 thresholds (-1.5% to -5%) × 4 holding periods = **28 total tests**.

Why this matters: multiple testing / "data snooping"

Here's the key insight, and it's one of the most important statistical ideas in the whole project: **if you run 28 independent tests, each with a 5% chance of falsely showing "significant" by pure chance, you should expect roughly 1-2 false alarms even if there's truly zero real effect anywhere.** This is called the **multiple testing problem**, and "data snooping" or "p-hacking" is what happens when a researcher runs many tests, finds the one lucky-looking result, and reports ONLY that one as if it were the plan all along.

What we found in the sweep: two cells did cross the $p < 0.05$ line (-2.5% threshold at 10-day hold, positive; -5% threshold at 3-10 day holds, negative). But neither is surrounded by a consistent pattern in neighboring thresholds — -2% and -3% right next to -2.5% show nothing, and -3.5%/-4% right next to -5% show nothing near as strong. **A real effect should show up smoothly as you nudge the threshold slightly; an isolated spike surrounded by nulls is the signature of noise, not signal.** This is exactly the "don't cherry-pick" lesson the assignment explicitly warns about, playing out in real numbers.

The honest, defensible move: report the whole 28-cell table, point at the pattern (or lack of one), and conclude no threshold gives robust evidence — instead of quietly only reporting the one good-looking cell.

Step 7: Out-of-sample validation

File: `src/oos_split.py`

We split the data in two:

- **Research period:** 2007-2019 (contains the 2008 crash)

- **Out-of-sample (OOS) period:** 2020–2026 (contains the 2020 COVID crash)

The rule that makes this valid: whatever threshold/holding-period/gap choices we locked in were decided using ONLY the research period. We then applied those exact same settings to the OOS period **without retuning anything**. If you're allowed to adjust your definition after peeking at the "test" data, the test proves nothing — this is the financial-research equivalent of a teacher handing out the exam answers before the exam.

What we found — this is the single strongest piece of evidence in the whole project:

- Research period: 1-day forward return is *significantly negative* ($p=0.024$) — meaning falls tended to CONTINUE falling for one more day, the opposite of "recovery."
- OOS period: 3-day forward return is *significantly positive* ($p=0.015$) — a different direction, a different holding period.

Neither the sign, the horizon, nor the significance carried over between the two periods. **If there were a real, stable recovery effect, you'd expect the same rough pattern to show up in both halves of the data.** Instead we got two different "significant" results pointing different directions — which is much stronger evidence of "no real effect" than a single non-significant p-value would have been on its own, because it shows the apparent pattern isn't even internally consistent.

Step 8: The simple backtest

File: `src/backtest.py`

Even though the statistical evidence says there's no real effect, the assignment asks for a simple backtest as an *extension*, not as a claim that this is a real strategy. Here's what it does and what every metric means:

- **Trade:** enter at `Open` the day after an independent event, exit at `Close` N days later (we used 5 days for this).
- **Costs:** 5 basis points (0.05%) charged on BOTH entry and exit, to simulate realistic slippage/brokerage. "Basis point" = 1/100th of a percent — a standard finance unit for small percentages.
- **Win rate:** % of trades that ended positive after costs.
- **Cumulative/total return:** what \$1 turns into if you took every trade in sequence, reinvesting the proceeds each time (compounding).
- **Max drawdown:** the single worst peak-to-trough decline in the cumulative equity curve — i.e., if you'd started tracking your account balance from its highest point so far, how much did it fall before recovering. This is a standard risk metric — a strategy can have a great total return and still be terrifying to actually hold if its drawdown is severe.
- **Cost drag:** how much of your gross return the transaction costs ate.

What we found: +34.5% net return over ~19 years sounds fine in isolation — until you see plain buy-and-hold NIFTY did +419% over the same span, and the strategy was only invested 5.8% of the time. That comparison isn't even fair in either direction (different risk exposure, different time horizons), which is exactly why we call it "context, not a benchmark" in the code and write-up. The -33% max drawdown comes almost entirely from a cluster of losing trades during the 2008 crash — several correlated bad trades in a row, the same clustering problem from Step 3 showing up again in a different form.

The correct framing, and the thing to say confidently on camera: the backtest exists because the brief asked for one, not because it proves anything. A positive-looking backtest number that isn't backed by statistical significance and doesn't survive out-of-sample is not evidence of a strategy — it's a number that happened to come out positive this one time, on this one dataset.

PART 3 — GLOSSARY (every term used, in plain language)

Term	Meaning
OHLC	Open, High, Low, Close — the four standard daily price points for any tradable asset.
Close-to-close return	$(\text{Close_today} / \text{Close_yesterday}) - 1$. The standard way to measure one day's price move.
Event	Our name for a day where the fall exceeds the chosen threshold.
Forward return	The return you'd get holding from an entry point to some number of days later.
Baseline	The same forward-return calculation applied to every day, not just event days — the "control group."
Look-ahead bias	Using information in your analysis that you couldn't have actually known at that point in time. Avoided here by entering at next-day open, not the event day's own close.
Independent event / clustering	Whether an event is far enough from the last one to be treated as a fresh, separate occurrence, vs. part of the same crash episode.
Basis point (bp)	1/100th of a percent. 5 bps = 0.05%.
Confidence interval (CI)	A range that likely contains the true average, given the noise in a limited sample. A 95% CI means: if you repeated this sampling process many times, about 95% of the intervals you'd construct would contain the true value.
Bootstrap	Estimating uncertainty by repeatedly resampling your own data with replacement, rather than assuming a theoretical distribution shape.
Permutation test	Testing whether a real difference between two groups is bigger than what you'd get by randomly reshuffling group labels many times.
p-value	The probability of seeing a result this extreme (or more) if there were truly no effect at all. Small p-value = the result would be surprising under "no effect."
Statistical significance	A p-value below a chosen cutoff (conventionally 0.05) — a convention, not proof of truth.
Multiple testing / data snooping / p-hacking	Running many tests and reporting only the good-looking one, which inflates the chance of a false positive.
Out-of-sample (OOS)	Data held back and not used to make any design decisions, used only afterward to check if a pattern found elsewhere still holds.
Overfitting	Tuning your method so precisely to one dataset that it captures noise specific to that data rather than a real, generalizable pattern.
Drawdown	The decline from a peak account value to the lowest point before a new peak is reached.
Win rate	The percentage of trades/events with a positive outcome.
Slippage	The gap between the price you expected to trade at and the price you actually got, usually due to execution delay or market impact.

PART 4 — EVERY COMMAND, IN ORDER

```
# one-time setup
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# full pipeline, run from src/
cd src
python get_data.py
python inspect_data.py
python event_engine.py
python stats_engine.py
python robustness_sweep.py
python oos_split.py
python backtest.py
```

Script	Output
get_data.py	../data/nifty_raw.csv
inspect_data.py	validation report (printed only)
event_engine.py	event detection + forward returns (printed only)
stats_engine.py	../results/stats_summary_all_events.csv , stats_summary_independent_events.csv
robustness_sweep.py	../results/robustness_threshold_sweep.csv , robustness_mingap_sweep.csv
oos_split.py	../results/oos_split_summary.csv
backtest.py	../results/backtest_trades.csv

Every parameter that changes behavior lives in `src/config.py` — you never need to touch logic in the other files to test a different threshold, holding period, or cost assumption.

PART 5 — LIKELY QUESTIONS AND HOW TO ANSWER THEM

“Why -3%?” No universally correct number exists for “significant fall” — we chose one that’s rare enough to be meaningful but frequent enough for a usable sample, then swept 7 alternative thresholds to confirm the conclusion doesn’t depend on this specific choice.

“Why does your headline result say ‘no effect’ — isn’t that a weak answer?” It’s an honest answer, and the brief explicitly says a negative result is acceptable — what matters is whether the investigation was done rigorously. We didn’t stop at one non-significant test; we ran a 28-cell robustness sweep and an out-of-sample split, both of which independently point the same way: no stable, repeatable effect.

“Couldn’t you have found a threshold that DOES work if you tried more combinations?” Almost certainly yes — and that’s precisely the trap. With enough tests, random noise eventually produces something that looks significant. The whole point of the robustness sweep was to show that any single “significant” result we might report is more likely a statistical fluke than a real pattern, given the entire table shows no consistent trend.

“What would change your mind — what evidence would make the hypothesis true?” A pattern that (a) holds across a reasonable range of thresholds, not just one lucky value, (b) has a consistent sign and rough magnitude across different time periods, and (c) survives being tested on data that wasn’t used to pick the definition in the first place. None of our results meet even the first bar convincingly.

“Is this backtest number (+34.5%) good?” No — and it’s explicitly framed as not meaningful in isolation: plain buy-and-hold returned +419% over the same period, the strategy was only invested 5.8% of the time, the underlying signal isn’t statistically significant, and it doesn’t survive the out-of-sample check. It’s included because the brief asks for a simple backtest as an extension of the research, not as evidence of a working strategy.

“What’s the single biggest limitation?” Sample size. Large market falls are inherently rare, so even after 19 years of daily data, we only have 59-79 independent events to work with — every confidence interval and p-value in this project should be read with that constraint in mind. That’s a limitation of the question itself, not a fixable flaw in the method.